第三讲教学设计：JSP核心技术—编写动态页面 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: JSP核心技术：编写动态页面
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet基础的Java Web开发初学者

一、教学目标

知识目标

1. 理解JSP（JavaServer Pages）作为视图层（View Layer）技术的根本目的：分离动态逻辑与静态显示。
2. 掌握JSP的三大核心语法：表达式Expression（`<%= ... %>`）、脚本片段Scriptlet（`<% ... %>`）和指令Directive（`<%@ ... %>`）。
3. 了解JSP的生命周期（Lifecycle），特别是它被Tomcat容器翻译成Servlet的核心原理。
4. 初步认识JSP九大隐含对象（Implicit Objects），重点理解`request`、`response`和`out`的直接可用性。

能力目标

1. 能够创建一个JSP页面，并嵌入简单的Java表达式来显示动态数据。
2. 能够在JSP页面中使用脚本片段（Scriptlet）实现简单的逻辑控制（如循环、判断）。
3. 能够使用`<%@ page ... %>`指令设置页面属性，如`contentType`和`import`。

素质目标

1. 建立"关注点分离"（Separation of Concerns）的软件工程思想。
2. 培养编写可读性、可维护性更强的视图代码的意识。
3. **[思政融合]** 介绍JSP技术时，可引申讨论自主知识产权技术的重要性，以及掌握核心技术对于国家信息安全和科技自立的意义。

二、教学重点与难点

教学重点

- JSP与Servlet在职责上的核心区别：JSP主外（负责展示），Servlet主内（负责控制）。
- JSP表达式（`<%= ... %>`）的简洁用法。
- `<%@ page ... %>`指令，特别是`contentType="text/html;charset=UTF-8"`的必要性。
- JSP的生命周期原理（JSP -> .java -> .class）。

教学难点

- **概念难点**: 真正理解JSP文件本身不是直接运行的，而是作为模板被服务器动态翻译成一个Servlet类来运行。
- **实践难点**: 在JSP中混合使用HTML标签和JSP脚本时，容易产生语法错误，需要理解代码的执行时机。

三、教学内容与时间安排

第一部分：问题引入 - 解救"意大利面条式"代码 (3分钟)

内容要点

1. **回顾痛点 (Recalling the Pain Point):**
   - 展示上一节课 `LoginServlet` 的代码，特别是 `out.println()` 中拼接大量HTML的部分。
   - **提问**: "同学们，请看这段代码。如果我们想给欢迎页面加一个复杂的表格、一些CSS样式，甚至一些JavaScript，会发生什么？我们的Java类会变成一锅混杂着Java逻辑、HTML结构、CSS样式的'意大利面条'！"
2. **场景冲突 (Scenario Conflict):**
   - **趣味故事**: "想象一下，一位UI设计师（小美）设计好了一个精美的HTML页面，她交给Java工程师（小明）。小明需要把这个HTML一行行地复制粘贴到他的 `out.println()` 语句里。第二天，小美说要改个颜色，小明又要在一大堆Java代码里找到那几行HTML去修改。这种合作方式，是不是一场灾难？"
3. **引出解决方案 (Introducing the Solution):**
   - **思路转变**: 我们能不能反过来？在一个HTML文件里，只在需要动态内容的地方"挖个坑"，然后用Java代码把数据"填进去"？
   - 这种"以HTML为中心，Java为点缀"的技术，就是 **JSP (JavaServer Pages)**。

教学方法

- 故事化教学，增强代入感。
- 对比法，强化JSP解决Servlet痛点的优势。

第二部分：第一个JSP页面 - 从静态到动态 (8分钟)

内容要点

1. **修改Servlet (Modifying the Servlet):**
   - 改造上节课的 `LoginServlet`，让它不再直接输出HTML，而是将数据准备好，然后"请求转发"给一个JSP页面去显示。

```
// In LoginServlet's doGet method
String agentName = request.getParameter("agentName");
// 将特工代号存入request这个"包裹"里，准备传递
request.setAttribute("agent", agentName);
// 请求转发给mission.jsp去显示
request.getRequestDispatcher("/mission.jsp").forward(request, response);
```

2. **创建第一个JSP (Creating the First JSP):**
   - 在 `webapp` 目录下创建一个 `mission.jsp` 文件。
   - **第一步：先写纯HTML结构**

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<html>
<head><title>任务简报</title></head>
```

```
<body>
    <h1>特工任务简报 (Mission Briefing)</h1>
    <hr>
    <h3>欢迎您，[这里应该是动态的特工代号]</h3>
    <p>您的任务书正在生成中...</p>
    <p>当前服务器时间是：[这里应该是动态的服务器时间]</p>
</body>
</html>
```

3. **填坑 - 使用JSP表达式 (Filling the Blanks with JSP Expressions):**
    - **获取数据**: 使用脚本片段（Scriptlet）从 `request` 中取出数据。 `<% String name = (String) request.getAttribute("agent"); %>`
    - **显示数据**: 使用JSP表达式 `<%= ... %>` 来显示变量。 `<h3>欢迎您，<%= name %></h3>`
    - **显示动态时间**: `当前服务器时间是：<%= new java.util.Date() %>`
4. **JSP生命周期揭秘 (Demystifying the Lifecycle):**
    - **核心原理**: 第一次访问 `mission.jsp` 时，Tomcat会在其内部工作目录里，把 `mission.jsp` 文件**翻译**成一个 `mission_jsp.java` 的Servlet文件，然后再编译成 `.class` 文件来运行。
    - 展示一张清晰的流程图: ** `mission.jsp` --> `mission_jsp.java` --> `mission_jsp.class` **。
    - **结论**: JSP本质上就是一种编写Servlet的便捷方式！它让我们专注于页面长什么样。

教学方法

- **代码重构**: 引导学生亲历从Servlet到JSP的改造过程。
- **原理图示**: 用图清晰展示JSP的底层工作原理，破除神秘感。

第三部分：JSP核心语法 - 动态能力的来源 (6分钟)

内容要点

1. **脚本片段（Scriptlets `<% ... %>`）：**
    - 用于编写任意Java代码块。
    - **趣味案例**: "假设总部发来了多条任务指令，我们需要用一个列表展示出来。"

```
<%-- 在JSP中创建一个任务列表 --%>
<%
    java.util.List<String> missions = new java.util.ArrayList<>();
    missions.add("任务一：学习JSP表达式");
    missions.add("任务二：掌握JSP脚本");
    missions.add("任务三：理解JSP生命周期");
%>
<h3>您的任务列表：</h3>
<ul>
<% for(String mission : missions) { %>
    <li><%= mission %></li>
<% } %>
</ul>
```

- **强调**: 这种写法虽然功能强大，但已经开始让HTML和Java代码再次混合，为下一课埋下伏笔。

2. **指令（Directives `<%@ ... %>`）：**
   - 是给JSP翻译引擎看的"指令"，用来设置整个页面的属性。
   - `<%@ page ... %>`：重点讲解 `contentType="text/html;charset=UTF-8"` （防止中文乱码）和 `import="java.util.List, java.util.ArrayList"` （导入Java类）。
3. **隐含对象（Implicit Objects）：**
   - 讲解JSP的一大便利之处：有9个可以直接使用的、预先定义好的对象，无需我们自己创建。
   - 重点介绍： `request` , `response` , `session` , `application` , `out` 。强调 `request` 和 `response` 就是从Servlet传递过来的那两个对象。

教学方法

- **功能驱动**: 以"显示任务列表"的功能需求来自然地引出循环脚本。
- **分类讲解**: 将JSP语法清晰地分为表达式、脚本、指令三类。

第四部分：总结与新的挑战 (3分钟)

总结要点

1. **JSP是"皮"**: 以HTML为中心，是视图模板。
2. **Servlet是"芯"**: 以Java为中心，负责逻辑控制。
3. **JSP的本质**: 它是一种能被自动翻译成Servlet的特殊文本文件。
4. **JSP的语法**: 表达式 `<%= ... %>` 负责输出，脚本 `<% ... %>` 负责逻辑。

提出新的挑战 (The New Challenge)

- 再次展示包含 `for` 循环的JSP代码。
- **提问**: "同学们，虽然我们把显示逻辑从Servlet移到了JSP，看起来好多了。但请看这里，我们的HTML页面里是不是又出现了 `<% for(...) { ... } %>` 这样的纯Java代码？如果设计师小美想调整这个列表的样式，她看到这些符号是不是又会头疼？"
- **展望**: "有没有一种更优雅、更像'标签'、而不是像'代码'的方式来完成这些循环和判断呢？当然有！下一节课，我们将学习EL表达式和JSTL标签库，它们将彻底把Java代码从我们的视图中驱逐出去，实现真正的'前后端分离'！"

四、教学方法与手段

教学方法

1. **故事化教学法 (Storytelling)**: 延续"设计师小美"故事线
2. **对比分析法 (Comparison)**: Servlet vs JSP的职责对比
3. **现场编码 (Live Coding)**: 演示JSP创建和运行
4. **原理图示法 (Diagram)**: 展示JSP生命周期
5. **双语教学法 (Bilingual Teaching)**:
   - 课堂采用英文授课，关键技术术语提供中英文对照
   - 重要概念（Expression, Scriptlet, Directive, Implicit Objects）首次出现时给出英文表达和中文释义
   - 鼓励学生使用英文术语描述JSP语法结构

- 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否理解Servlet请求转发的作用；能否正确使用JSP表达式输出变量。
- **终结性评价**: 能否用自己的话解释JSP和Servlet的区别与联系；能否独立编写一个简单的JSP页面来显示动态信息。

六、课后拓展

思考题

1. 既然JSP最终会变成Servlet，为什么我们不直接写Servlet呢？JSP的主要优势体现在哪里？
2. 在JSP页面顶部，`<%@ page contentType="text/html;charset=UTF-8" %>`这行代码如果漏掉了，可能会发生什么问题？为什么？

实践作业

1. 创建一个`product.jsp`页面。
2. 在Servlet中创建一个`Product`对象（一个简单的Java类，有`name`和`price`属性），并设置其值。
3. 将这个`Product`对象通过`request.setAttribute()`传递给`product.jsp`。
4. 在`product.jsp`中，使用JSP表达式和脚本，以表格的形式显示出产品的名称和价格。